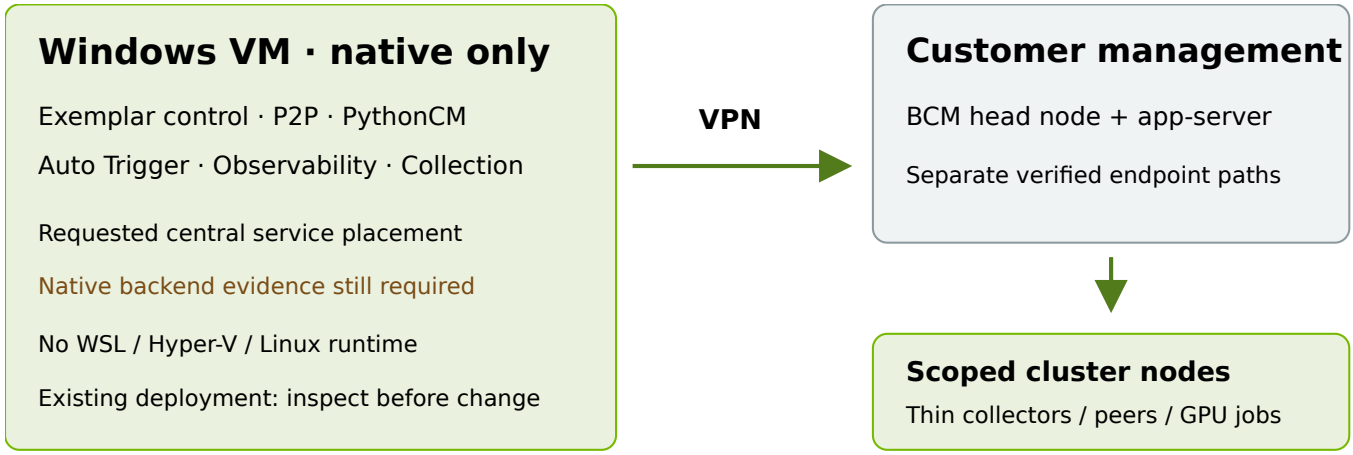
NVIDIA · OVA OPERATIONS

OVA Monorepo Windows

Deployment and operator guide

Six coordinated tools · Windows preparation · VPN access
Service placement · Offline delivery · Acceptance and recovery



Original release: 20261105-windows-native-02
Guide date: 6 November 2026
Read the deployment boundary and validation report before installation.

Contents

From verified source to a measured, recoverable deployment.

1. What is included

2. Intended topology and placement decision

3. Responsibilities and site inputs

4. Release integrity and payload selection

5. Prepare the Windows VM

6. Configure VPN and prove both network paths

7. Offline staging and transfer

8. Commission the six components

9. Acceptance evidence and release gates

10. Daily operations

11. Troubleshooting

12. Updates, rollback and handover

13. Source and reading order

14. Coordinate with the existing Windows agent

Deployment and operator guide

Release basis: `20261105-windows-native-02` · Guide date: 6 November 2026

This guide explains the six-tool delivery, Windows preparation, VPN connectivity, backend placement, commissioning, everyday operation and recovery. It is intended for the operator of an ITSS-provided Windows VM or another approved Windows VM on the NVIDIA internal network.

Read the placement statement before installation. The requested target is a central Windows VM hosting most or all application services, with only thin clients and collectors on the customer cluster. The retrieved revision02 release implements Windows staging, SSH orchestration and browser access; its application backends run on designated remote Linux management hosts. It expressly assumes no WSL, Hyper-V or Linux Docker engine on Windows. Central Windows runtime hosting is not established by that release. This publication adds a separate PythonCM source snapshot; that addition alone does not establish Windows runtime compatibility. Those are engineering and platform decisions that must be resolved and validated before claiming the requested architecture is delivered.

A repository checkout, package checksum or successful Linux source test does not establish a Windows runtime pass. Consult the repository's validation report for the actual tests run in this publication and the native Windows compatibility audit for concrete source blockers. That audit is a source analysis; it does not establish the installed state of the already deployed Windows VM.

1. What is included

Component	Function	Current release runtime boundary
Exemplar Recipe	Prepare pinned NVIDIA recipes, staged images/models and bounded GPU validation jobs	Windows can manage files and invoke SSH; Linux controller and selected GPU nodes execute Slurm/Enroot/Pyxis workflows
P2P	Distribute staged assets through Dragonfly Manager, scheduler, seed and scoped peers	Linux management runtime and node peers; Windows browser uses an SSH tunnel
Rack Status	Inventory, health UI and approved remote cluster actions	Requested PythonCM source is added separately; original release retains `nvis/nvis-rack-status` with Linux backend, web UI and optional desktop shell
Auto Action Trigger	Monitor and collect results; optionally launch single-node tests and recovery actions	Linux/Slurm scripts; all action/recovery switches remain disabled for commissioning
Observability	Metrics, logs, dashboards and source discovery	Linux core services; thin exporters/Alloy near data sources; Windows browser access
OVA Collection Framework	Collect durable scheduler evidence and declared artifacts	Linux collection service/hooks near approved sources; persistent state and output storage

The six components retain independent runtimes, configuration, lifecycle and acceptance requirements. The bundle coordinates their delivery; it does not turn them into one Windows executable.

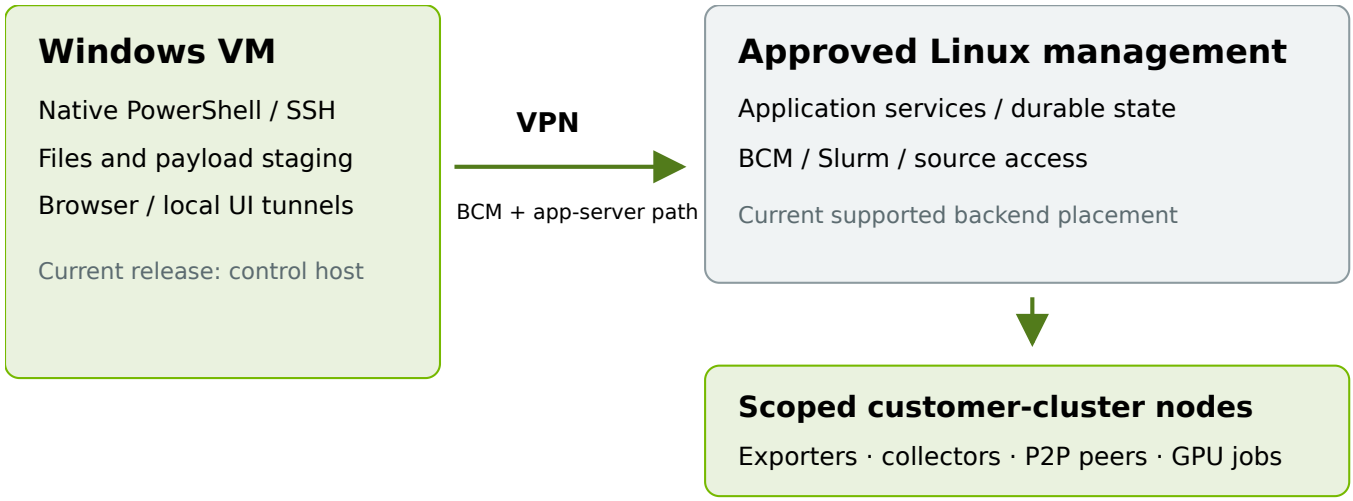
Rack Status source identity

The current delivery deliberately includes `nvis/nvis-rack-status`, rather than `nvis-ova/rack-status-pythoncm`. This monorepo adds `nvis-ova/rack-status-pythoncm` at commit `8d1896e55cf24c23ef80d9ccef8a0d9a8a7962f7` in `components/rack-status-pythoncm`. Validate its API, UI, BCM access and Windows compatibility before selecting it for deployment. The original Rack source is retained as a historical release component, not a silent replacement for PythonCM. A desktop shell does not prove native backend compatibility. Keep both source identities explicit in the software inventory.

2. Intended topology and placement decision

The requested topology places the management applications on the Windows VM. The cluster retains its existing BCM head node and app-server, and receives only scoped collectors, P2P peers and workload execution artifacts. GPU work and cluster-local data transfer stay within the customer network.

The current delivery topology is different:




```
NVIDIA internal network             VPN                Customer network
Windows VM / native PowerShell  --------->  approved SSH entry / management host
  release and payload staging               Linux application services + state
  checksum verification                      BCM head node / app-server access
  SSH/SFTP control                            |
  browser and optional shell                 +-- scoped exporters / collectors
  localhost SSH forwards                     +-- P2P peers / scheduled GPU jobs
```

The user requires native Windows only: no WSL, Hyper-V, Linux VM or Linux container engine. Central services must therefore use genuinely Windows-compatible implementations or be ported and tested. An earlier deployment agent reportedly installed the package on the actual Windows VM; its installed state and launch commands must be inspected before changing architecture or duplicating services. A successful SSH launcher does not prove the backend is native. Record each executable, host, service owner, state directory and endpoint.

The original Linux management deployment remains documented here as the historical release behavior, not as an approved substitute for the requested native Windows architecture. Do not deploy new remote heavy services to satisfy the native-only requirement. Inspect the existing Windows installation and record remaining native compatibility gaps explicitly. Cluster-side collectors and workload execution remain within the approved thin-agent scope.

3. Responsibilities and site inputs

Owner	Required information or action
ITSS / Windows VM owner	VM access, Windows version/architecture, approved software, disk quota, account permissions and supported virtualization policy
VPN / network owner	Fortinet client and private instruction file, gateway/profile, authentication/MFA, routes, DNS, proxy policy and access to both BCM and app-server
Cluster owner	Approved SSH identity and fingerprints, management host placement, BCM/app-server names, Linux platform, collector scope and scheduler access
Tool operator	Release verification, compatible payload selection, reviewed configuration, installation receipts and bounded canary checks
Service owner	Persistent storage, backup/restore, authentication, retention, restart policy and update window

Collect private values in an access-controlled site plan outside Git. Include Windows OS/build and CPU architecture; approved software; free staging and runtime disk; SSH aliases/user/ports; trusted host-key fingerprints; BCM and app-server roles; backend placement; Linux OS/CPU/Python ABI; existing container runtime; source paths; listener ports; credentials by file reference; retention; and the permitted node/partition/GPU/time scope.

Do not guess VPN gateway, address space, credentials, DNS suffixes or route rules. Use the supplied private VPN instructions. NGC/Hugging Face access is needed only for selected protected workload assets; it is not required for the basic public Dragonfly console.

4. Release integrity and payload selection

Use the cleaned publication artifact `artifacts/OVA-MONOREPO-WINDOWS-control-sanitized-20261106.zip`. Read its exact bytes and SHA-256 from `evidence/sanitized-artifact-provenance.json` in the same checkout; removed runtime credential files are recorded by path in `evidence/sanitization-removals.json`. The sanitized artifact is the primary extraction/staging source. Do not use the original r2 ZIP for installation or distribution.

The historical source release index identifies these original inputs for audit only:

Artifact	Bytes	SHA-256
`IREN_MKZ-windows-control-20261105-r2.zip`	636,211,960	`28765b63d5331dc1413a43152c433af288897059960d7f636385bb78247607e7`
Observability `linux-amd64` dependency ZIP	680,949,898	`914991007abbbef5a66f00a01864c8c8715ed4ef5bfc110c964bc66e46fe4519`
Observability `linux-arm64` dependency ZIP	634,417,483	`1386ed791e5879d5071f0d1c97a912255c91b36052e47ec1481bc2c07a5904e3`

These are original source-artifact hashes, not hashes of a revised Git checkout or a newly generated repository archive. A credential scan identified generated private credential material in the original packaged Rack source; publication removes it and records a sanitized derivative. Use the publication manifest for newly packaged artifacts. Never redistribute the unreviewed original as if it were the cleaned release. A retained vendor PostgreSQL image contains the standard Debian example `ssl-cert-snakeoil.key`; it is a vendor default, not an issued site identity. Configure fresh site-generated TLS keys/certificates for any deployed TLS service and never use that example key as the site certificate identity.

The historical original control ZIP was retrieved from `IREN_MKZ/bundles/20261105-windows-native-02/`; do not select it as the installation source. Obtain the sanitized artifact from this repository publication. Observability dependency objects deliberately retain their revision01 paths under `IREN_MKZ/bundles/20261105-windows-native-01/tools/observability/`; revision02 reuses the same verified bytes. These Linux dependency packs describe the historical release path and cannot execute natively on Windows. Where reproducing that path for analysis, select the dependency architecture of the Linux backend, regardless of the Windows host architecture. A Windows x64 VM may control a Linux arm64 backend.

The control ZIP does not contain the large Observability platform packs. A Git repository containing the control sources alone is not a complete offline installation medium. Reconcile the exact dependency inventory for every selected function, including OS/runtime prerequisites and optional workload assets.

5. Prepare the Windows VM

Use short, separate directories. The following layout is an operator convention, not an installer requirement:

```
C:\OVA\downloads\          immutable incoming archives
C:\OVA\control-sanitized\         verified control bundle, kept unchanged
C:\OVA\payloads\           external images, wheels and binary packs
C:\OVA\work\               unpacked working toolkits
C:\OVA\private\            private site plan and credential references
C:\OVA\evidence\           logs, receipts and acceptance results
```

Record Windows version/build, PowerShell version, free storage, OpenSSH tools, Python and browser availability. The bundled `Test-WindowsControlHost.ps1` checks the control host; inspect its report and resolve missing tools. It does not install Linux backends.

Obtain the sanitized control ZIP from the published repository artifact path and compare its hash with `evidence/sanitized-artifact-provenance.json`. Use the authenticated NVFTP interface only for separately approved external dependency packs and preserve their recorded hashes. Use individual credentials from the approved credential store. Verify the sanitized control ZIP before extraction:

```
$Publication = "C:\OVA\repository" # actual checkout location
$Provenance = Get-Content -Raw (Join-Path $Publication "evidence\sanitized-artifact-provenance.json") |
ConvertFrom-Json
$Archive = "C:\OVA\downloads\OVA-MONOREPO-WINDOWS-control-sanitized-20261106.zip"
$ActualHash = (Get-FileHash -Algorithm SHA256 $Archive).Hash
if ($ActualHash -ne $Provenance.sanitized_control.sha256) { throw "Sanitized archive hash mismatch" }
if ((Get-Item -LiteralPath $Archive).Length -ne $Provenance.sanitized_control.bytes) { throw "Sanitized archive
size mismatch" }
Expand-Archive -LiteralPath C:\OVA\downloads\OVA-MONOREPO-WINDOWS-control-sanitized-20261106.zip -DestinationPath
C:\OVA\control-sanitized
Set-Location C:\OVA\control-sanitized
py -3 scripts\verify_bundle.py .
.\scripts\Test-WindowsControlHost.ps1
```

The example checks the exact SHA-256 and length against the publication provenance record; the original historical hash is different. Stop on mismatch. Extract into a fresh directory. The strict `FILE_MANIFEST.json` describes the control tree; place working toolkits and external payloads outside it so ordinary work does not invalidate verification.

If native Python is missing, `windows/README.md` describes an optional official CPython embeddable x64 interpreter for standard-library verification helpers. Extract it outside the verified tree and call its full executable path. That interpreter does not provide pip, Linux packages or application services.

6. Configure VPN and prove both network paths

1. Read the private Fortinet instruction file supplied by the network owner. Install or select the approved client/profile and follow the documented authentication/MFA flow.
2. Record the pre-VPN network state. Connect the VPN and inspect the assigned adapter, approved routes and DNS resolution. Do not widen routes or disable certificate validation to force access.
3. Resolve both the BCM head node and the app-server. Test each endpoint and the exact service port from the site plan.
4. Establish SSH using the approved user/key and independently verified host-key fingerprint. Test app-server and head-node access separately; success to one does not prove the other.
5. Check whether VPN policy prevents access to NVIDIA services. Prestage all required packages before connecting when necessary.
6. Confirm how services in the proposed Windows-side runtime would reach the same endpoints if central hosting is selected. Host-level reachability does not prove a nested runtime has the required route or DNS.

Read-only diagnostic examples use placeholders that must be replaced with site values:

```
Get-NetIPConfiguration
Get-NetRoute -AddressFamily IPv4
Resolve-DnsName <BCM_HEAD_NODE>
Resolve-DnsName <APP_SERVER>
Test-NetConnection <BCM_HEAD_NODE> -Port <APPROVED_SSH_PORT>
Test-NetConnection <APP_SERVER> -Port <APPROVED_SERVICE_PORT>
ssh <APPROVED_SSH_ALIAS> hostname
```

Keep private addresses, route output and identities in the private evidence directory. Run the supplied remote Linux preflight using the parameters shown by its help/source; record OS, architecture, available storage and runtimes. Inspect existing listeners and service names before assigning ports.

UI access by local SSH forward

Windows can initiate a local loopback tunnel through the VPN to a remote loopback web service. This avoids requiring a callback from the customer cluster into Windows for browser access:

```
ssh -N -L 127.0.0.1:<LOCAL_UI_PORT>:127.0.0.1:<REMOTE_UI_PORT> <APPROVED_SSH_ALIAS>
```

Keep the SSH session open while using the UI. Select distinct local ports and verify the TLS identity. Changing only a local port cannot resolve two backend services bound to the same remote port. Browser access and telemetry delivery are separate network flows: an SSH UI forward does not configure exporter push routes.

7. Offline staging and transfer

Choose the backend OS/CPU/ABI before selecting wheels or binaries. Windows Python wheels do not install into Linux, and Linux wheels do not make a Windows service native. Record archive hashes and platform compatibility for each dependency.

Use the supplied transfer helper to create a new versioned remote staging directory and verify the transferred ZIP:

```
# Use $Provenance loaded from the matching publication in section 5.
.\scripts\Send-Release.ps1 -SshAlias <APPROVED_SSH_ALIAS> `
  -BundleZip C:\OVA\downloads\OVA-MONOREPO-WINDOWS-control-sanitized-20261106.zip `
  -ExpectedSha256 $Provenance.sanitized_control.sha256 `
  -ReleaseId <UNIQUE_RELEASE_ID>
```

This copies and verifies; it does not extract or install. Preserve LF endings and executable modes for Linux scripts. On the designated Linux staging host, extract the control tree, verify it and unpack selected tool ZIPs to separate working directories using the supplied `unpack_toolkit.py` helper. Keep external Observability payloads in a sibling payload directory and verify both the outer archive and its internal inventory.

A URL in a lock file is not an offline payload. Reconcile every selected dependency before entering an environment with no internet access. Container images do not supply a host driver, a container engine, site certificates or OS packages automatically.

8. Commission the six components

This section records the historical release commissioning procedure for reference. It is not authorization to move backends to Linux or enable virtualization. Under the confirmed native-only requirement, first inventory the deployed Windows services with section 14; execute only the steps compatible with the measured native implementation and approved site scope. Record incompatible Linux installation steps as gaps.

The order below reduces confusion during initial acceptance. It is an operational sequence; it is not a claim that every tool depends on every preceding tool.

8.1 Establish management foundation

Confirm approved service placement, trusted SSH, persistent storage, time synchronization and compatible runtimes. Create separate working/state/log locations. Confirm ownership and backup destinations. Record existing jobs/services before changes. Keep cluster inventory and credentials out of the repository.

8.2 Observability

Read the included Observability handoff and dependency manifest. Load the exact platform-matched images/binaries/plugin/wheels using its documented offline route. Start with explicit empty discovery and verify authenticated dashboard access. Add one approved exporter/source, then check timestamps, labels, metric/log arrival, error logs and the intended node identity.

The current stack uses Linux services such as Prometheus, Loki, Grafana and Alloy. Do not run its Bash setup script in native PowerShell as if it were a Windows installer. A central Windows-hosted deployment needs its own validated runtime plan. Exporters remain near the data they observe. If pushing to a Windows-side endpoint, prove the approved return route and receiver access; a VPN initiated by Windows does not inherently grant cluster-initiated inbound connectivity.

8.3 Rack Status

Read the exact delivered source identity and configuration reference. Configure the approved BCM/SSH/API endpoints and prebuilt SPA using its own handoff. Verify authentication, read-only inventory, node naming and freshness first. Keep disruptive actions and automated workflows disabled until scoped commissioning succeeds.

Revision02's Rack Status has an optional Windows desktop shell. The shell is a client and does not establish native compatibility of the Flask backend, Unix locking or Linux management commands. If PythonCM is selected instead, record the replacement source/version and repeat backend, authentication and cluster integration acceptance for that implementation.

8.4 OVA Collection Framework

Configure a minimal explicit source set before applying the deployment plan. Applying the provided plan performs collection passes and enables timers/services; the release does not provide a paused installer. Preserve its durable state and output data when updating.

Register exact approved config/validation paths and the intended cluster/epoch identity. Establish durable scheduler completion/accounting and source retention before the first scoped job. Confirm file permissions, exact-byte capture, one restart/replay path and independent reconciliation against scheduler inventory. The collector cannot recover deleted source bytes. Arrays, heterogeneous/federated jobs and oversized/unstable remote files require the adapters and limits described in its source documentation.

8.5 Basic P2P

Use the dedicated minimal Dragonfly profile. The documented chain is PostgreSQL 16 and Redis, then Manager, scheduler, seed and a scoped peer, with the web console/API behind nginx HTTPS. Read the included P2P `agent_handoff.md` and use its ordered render, preflight, image-load and console-check helpers.

The release's default console is remote loopback HTTPS port `4301`; Manager REST port `12004` is a separate backend endpoint. Forward the console to Windows and verify/trust the certificate using the kit procedure. The supplied initial login is `root` / `dragonfly`; change it at first login. This bootstrap web account is distinct from SSH, database and optional object-store identities.

Use a generated synthetic file and a controlled internal origin to verify bytes at a seed and one peer. Record hashes, transfer evidence and actual topology. The preparation canary retained a one-byte origin metadata probe; keep the approved internal origin available. Small-file behavior may differ. Basic core acceptance needs no NGC token, MinIO, model or GPU allocation.

Do not invoke Rack's older `nvis-p2p-provision` action to install this minimal profile. Its advanced layout and artifact lock are different, and the release carries an explicitly patched Manager derivative. Do not overwrite that derivative with an older locked binary. Qualify Rack-to-P2P endpoint integration separately after both services pass independent acceptance.

The optional dfstore/MinIO/full-object-storage path has additional dependencies and its own handoff. Historical NVIDIA workload catalog/receipt hash disagreements remain gates for the affected payloads; synthetic-file success does not resolve them.

8.6 Auto Action Trigger and single-node tests

Install and configure from its own handoff. The supplied commissioning profile leaves all 13 action/recovery switches disabled. Verify paths, read-only monitoring and result collection before enabling any loop.

For each single-node test define an explicit node, scheduler partition/account, GPU allocation, maximum duration, exact command, output location and cleanup behavior. Inspect the proposed action before execution. Keep drain clearing, daemon restarts and GPU jobs off until that scope is supplied. Record job IDs and distinguish workload failure from collection or transport failure.

8.7 Exemplar Recipe

Select a supported recipe for the actual GPU architecture, precision, scheduler and scale. Pin the official upstream commit and stage the matching container/model assets. Use the included preparation kit and its validation commands; the kit JSON is not a replacement for the upstream CLI's release-specific configuration.

Exemplar's current runtime route uses Linux, Slurm, Enroot/Pyxis and platform-specific GPU/RDMA software. Windows manages preparation/control through approved remote execution. Validate controller access, container launch, per-node assets, fabric and scheduler health before submitting a canary. Use only the requested node/partition/GPU/time limits, retain all outputs and use hardware-specific acceptance criteria. A scheduler exit code alone does not establish performance qualification.

9. Acceptance evidence and release gates

Gate	Evidence required	What it establishes
Source integrity	Measured source archive hash, exact component revisions, file manifest verification	The intended bytes were staged
Windows control	Actual Windows OS/build, PowerShell/Python/OpenSSH results and helper execution logs	Control operations work on that VM
Central Windows hosting	Actual service startup/restart, runtime architecture, persistence and all endpoint flows from Windows-side runtime	Requested central placement works for each tested component
VPN paths	Separate DNS/port/trusted SSH results to BCM and app-server; telemetry route results where needed	The required network paths work
Backend offline install	Matching platform, no unintended network fetch, package/import/service results	Selected runtime dependencies install offline
Component canary	Authenticated UI or bounded workload plus exact outputs/bytes and expected error handling	That component's scoped flow works
Recovery	Service restart, VPN interruption and retained-source replay/backup restore evidence	Tested recovery behavior works
Site acceptance	Named service owners, actual node coverage, unresolved gaps and agreed acceptance criteria	The stated operational scope is accepted

Use precise status labels: `STAGED`, `OFFLINE_INSTALL_VERIFIED`, `SERVICE_READY`, `CANARY_PASSED` and `SITE_ACCEPTED`. Each requires its own evidence. Never translate preparation tests into unrun Windows, VPN or customer-cluster passes.

Current source documentation reports preparation tests and Linux offline checks; it explicitly says no Windows VM or customer target was contacted during preparation. This guide does not certify that all six native Windows backends have passed. A Windows CI runner test establishes only the controls and code paths recorded in its report; the actual VM installation needs its own measured evidence. Use the publication's validation record for any additional evidence gathered after retrieval.

10. Daily operations

Before starting a session, verify the intended VPN profile and target SSH aliases. Check service health, disk usage, retention and collector freshness. Open UI tunnels to the approved endpoints and inspect the selected cluster identity before any action. Keep jobs, distribution campaigns and remediation bounded to the reviewed scope.

After a session, record jobs and operations left running, close local tunnels and disconnect VPN when appropriate. Disconnecting VPN does not stop remote services, collectors or jobs. If services are hosted on the Windows VM, VM shutdown and runtime restart policies determine their availability; test them explicitly before operational handover.

Retain private runtime configuration separately from immutable application source. Back up collection state, dashboard configuration, time-series/log data according to retention requirements, P2P metadata needed for recovery and operator receipts. Backups must be restorable; record a tested restore into an isolated destination.

11. Troubleshooting

Symptom	Check first	Next action
NVFTP fails after VPN connection	VPN proxy/route policy and pre-staged payload inventory	Use the approved access path or stage packages before VPN; retain checksum verification
BCM works, app-server fails	Separate DNS, route, port and identity for app-server	Resolve the specific path with the network/cluster owner
SSH host-key prompt changed	Independently supplied fingerprint and intended hostname	Resolve identity before continuing; do not disable checking
UI unavailable	Tunnel process, remote service state, remote listener and local port collision	Verify service locally on backend, then reconstruct the correct tunnel
P2P console login fails	Correct console endpoint, bootstrap password state and patched Manager provenance	Use minimal-kit diagnostics; do not replace its binary with the legacy Rack lock
Offline install tries internet	Wrong platform wheelhouse, missing dependency or absent offline flag	Reconcile exact inventory and documented offline options
Metrics stale	VPN direction, exporter/Alloy logs, receiver endpoint, time and labels	Trace a single scoped source end to end
Collection gaps	Source retention, permissions, accounting/completion records, identity and size limits	Preserve sources and replay; report unresolved gaps explicitly
Wrong node appears	Retained example inventory, discovery labels or wrong cluster alias	Correct private configuration before expanding coverage
Native Windows import/command fails	Unix-only dependency or Linux executable	Use an approved validated runtime or complete the required port; record the compatibility failure

12. Updates, rollback and handover

Stage each new release beside the old one and verify its manifests before changes. Compare source revisions, runtime dependencies and configuration schema. Back up persistent state and record current service/configuration paths. Update one scoped component first and rerun its acceptance flow before expanding.

Rollback application binaries/configuration only through the component's supported procedure. Do not assume a database/state schema can be downgraded. Restore an appropriate backup into an isolated destination when required. Preserve collector identity, checkpoints and captured payloads; do not reset state or job IDs to hide gaps.

The private handover report should include release ID and hashes; component repositories/commits; Windows OS/build; exact backend placements and runtime versions; approved endpoint aliases; service/listener/tunnel mappings; configuration and credential file locations without values; state and backup paths; collector/node coverage; test/job IDs; measured results; skipped checks; known limits; rollback procedure; and responsible service owners.

13. Source and reading order

The working control delivery is under `control/`; unpacked component sources and the additional PythonCM snapshot are under `components/`. This guide and its PDF are under `docs/`. Run root helper commands from the extracted sanitized control bundle or from `control/` in the checkout, not from the repository root. Keep the verified `control/` tree unchanged after its publication manifests are generated. Provenance records distinguish the original source hashes from any sanitized derivative published here.

Start with the published repository README and validation report, then the original release `agent_handoff.md`, `RELEASE_INVENTORY.json`, `DEPENDENCIES.json`, `FILE_MANIFEST.json` and `docs/artifact-index.md`. Read `docs/windows-vpn-runbook.md` and `docs/airgap-and-acceptance.md` before site work. Each component has its own ZIP and handoff; those exact versioned instructions determine installation commands and supported options.

Original distribution: NVFTP IREN_MKZ.

Requested repository destination: nvis-ova / OVA-MONOREPO-WINDOWS. Publication status and commit identity belong in the final validation record; this link alone is not proof of a completed push.

For executable commands, use the source files from the checkout. The PDF is an operator reference; placeholders must be replaced with approved private site values before execution.

14. Coordinate with the existing Windows agent

The user has already deployed through another agent on the Windows VM and requests direct agent coordination. Use the configured Codex app-server endpoint and target thread if available. Do not create a GitLab issue, merge request or repository file as an agent message transport. Publishing the source repository is a separate task.

The local CLI supports a remote queue interface; inspect `codex queue --help` on the coordinating host and use its documented `--remote` endpoint and thread arguments for that installed version. Do not guess the app-server address, expose it publicly or weaken its access controls. The actual endpoint/thread or an approved remote execution route is required before a message can be delivered. Preserve the existing agent's state and request read-only evidence first.

For local CLI version 0.166.0, the inspected help establishes this form (replace placeholders and provision the named token environment variable through the approved secret store):

```
codex queue --remote <APP_SERVER_ENDPOINT> `
  --remote-auth-token-env <TOKEN_ENVIRONMENT_VARIABLE_NAME> `
  --thread <EXISTING_THREAD_UUID_OR_EXACT_NAME> `
  --message "Please run the agreed read-only OVA native Windows inventory."
```

`--resume` is optional: it attempts to resume that thread on the selected server. Queueing a message and resuming execution are separate operations; retain existing thread ownership and do not start duplicate installation work.

Suggested first message to the Windows agent:

```
Inspect the existing OVA installation on this Windows VM. The user requires
native Windows only: no WSL, Hyper-V, Linux guest or Linux container runtime.
Do not reinstall or move services. Report Windows OS/build and the installed
package/revision. For each of Exemplar, P2P, Rack Status PythonCM, Auto Action
Trigger, Observability and OVA Collection Framework, report executable path,
launch command, actual execution host/OS, process/service state, listener,
configuration path, state directory and dependencies. Redact secret values.

Identify the actual Rack repository/commit and whether RACK_SCHEDULER uses
pythoncm. Describe VPN connectivity separately to BCM head node and app-server;
use the supplied private Fortinet instructions and existing trusted SSH aliases.
Run the read-only inventory helper from the published checkout:

  .\scripts\Get-NativeWindowsDeploymentEvidence.ps1 `
    -OutputDirectory <FRESH_PRIVATE_EXTERNAL_DIRECTORY> `
    -BcmHeadNode <APPROVED_BCM_HOST> -AppServer <APPROVED_APP_SERVER>

Run read-only native helper and application smoke checks only. Distinguish a
Windows client from its backend and distinguish historical Linux fixture checks
from an actual Windows run. Return logs/results through this app-server thread.
Keep remediation, drain changes, new workloads and service moves disabled.
```

Native capability record to complete on the VM

Capability	Source evidence	Required actual-VM result
Windows verification, PowerShell, SSH/SFTP and UI access	Native control helpers are supplied	OS/build, helper execution, hash verification and tunnel/UI log
Rack Status PythonCM backend	Requested source snapshot added; PythonCM provider exists	Native import/start/auth/read-only BCM check; prove no hidden Linux backend
P2P Manager/scheduler/seed/peer	Delivered minimal profile uses Linux assets	Identify native Windows binaries and support; prove synthetic bytes through actual peers
Observability core	Delivered image/binary packs target Linux	Identify native service builds, persistence and authenticated metrics/log ingestion
Collection controller	Delivered Linux service/hooks and portable preparation checks	Native state/storage/collection path or explicit missing port; exact-byte recovery evidence
Auto Trigger coordinator	Delivered Linux/Bash/Slurm scripts	Identify native coordinator and read-only monitoring path; retain disabled actions
Exemplar preparation/control	Portable preparation helpers plus Linux workload route	Native helper results and bounded remote execution path; GPU workloads remain on cluster

Source availability is not a native backend pass. The final installed-state report must resolve every row with measured evidence or an explicit remaining gap.